```
int main()
{
80483e4:       55                      push   %ebp
80483e5:       89 e5                   mov    %esp,%ebp
80483e7:       83 e4 f0                and    $0xfffffff0,%esp
80483ea:       83 ec 20                sub    $0x20,%esp
    int i = 10;
80483ed:       c7 44 24 18 0a 00 00    movl   $0xa,0x18(%esp)
80483f4:       00
    int size_of_i = sizeof(i++);
80483f5:       c7 44 24 1c 04 00 00    movl   $0x4,0x1c(%esp)
80483fc:       00
    printf("Size of i : %d\n", size_of_i);
80483fd:       b8 00 85 04 08          mov    $0x8048500,%eax
8048402:       8b 54 24 1c             mov    0x1c(%esp),%edx
8048406:       89 54 24 04             mov    %edx,0x4(%esp)
804840a:       89 04 24                mov    %eax,(%esp)
804840d:       e8 ce fe ff ff          call   8048300 <printf@plt>
    printf("value of i : %d\n", i);
8048412:       b8 10 85 04 08          mov    $0x8048510,%eax
8048417:       8b 54 24 18             mov    0x18(%esp),%edx
804841b:       89 54 24 04             mov    %edx,0x4(%esp)
804841f:       89 04 24                mov    %eax,(%esp)
8048422:       e8 d9 fe ff ff          call   8048300 <printf@plt>
    return 0;
8048427:       b8 00 00 00 00          mov    $0x0,%eax
}
804842c:       c9                      leave
```

Figure 4: Assembly code generated by the compiler for Code 3

**Code 3**

```
1 #include <stdio.h>
2 #include <stddef.h>
3
4 int main()
5 {
6         int i = 10;
7
8         size_t size = sizeof(i++);
9
10        printf("Size of i : %lu\n", (long unsigned )
    size);
11
12        printf("value of i : %d\n", i);
13
14        return 0;
15 }
```

Can you guess what the output of the above mentioned program will be? At first glance, anybody would say it is 4 (assuming the *sizeof(int)* is 4 bytes) and 11. But, when I run the program in my system, it shows 4 and 10 (refer Figure 3 for output).

Why are we getting the value of variable 'i' as 10 instead of 11? Here is the reason.

The *sizeof* operator is the only one in C, which is evaluated at compile time, where *sizeof(i++)* in our example is replaced by the value 4 during compile time itself. We can validate this by referring to Figure 4, which contains the assembly code equivalent to the *sizeof(i++)* in C.

**Note:** To obtain the assembly code as shown in Figure 4, follow the steps shown below:
+ *gcc -g filename.c* (in our case, the file name is *sizeof_run.c*)
+ *objdump -S output_file* (in our case, the output file is *a.out*)

From Figure 4, we can see that *sizeof()* is completely evaluated at compile time (the exception is *gcc*, which supports zero-sized structures as a GNU extension, which is evaluated at the runtime). And the whole *sizeof(i++)* is replaced by the constant value 4, which is highlighted in the box. Hence, there are no assembly instructions for *i++* at all, which is supposed to be evaluated at the runtime.

## Case 2: Runtime behaviour

As mentioned earlier, *sizeof()* is the only operator in C, which is evaluated at the compile time. But, there is an exception for this in C99 standards, for variable length arrays.

To start with, let us consider the following code (Code 4):

**Code 4**

```
1 #include <stdio.h>
2 #include <stddef.h>
3
4 int main()
5 {
6         unsigned int size;
7         size_t array_size;
8
9         printf("Enter the size:");
10        scanf("%u", &size);
11
12        //Declaring the variable length array
13        int array[size];
14
15        //Finding the size of array
16        array_size = sizeof(array);
17
18        printf("Size of array : %lu\n", (long unsigned )
    array_size);
19
20        return 0;
21 }
```

Let us see the output, when the above code is compiled and run (shown in Figure 5).

From the above output it is very clear that the *sizeof()* operator is evaluated at runtime. We can observe the equivalent assembly code generated by the compiler as shown in Figure 6.

Also, note the difference between the assembly code in Figures 4 and 6.

# The need for *sizeof()*

## Case 1: Auto determination of the number of elements in an array

To compute the number of elements of the array automatically, depending on the data-type of the element, the *sizeof()* operator comes in handy.